

业务背景-长视频APP潜在付费人群挖掘

当前业务会员增长停滞，需要寻找新的增长机会点。在会员增长场景中，试用会员是核心转化手段之一。但试用并非随机分配，而是基于用户历史行为进行策略触达，直接使用转化率对比会存在显著选择偏差。

同时，由于会员转化存在明显的时间趋势和活动影响，单纯的 before-after 对比也难以识别真实因果效应。

在无法立即开展大规模 AB 实验的前提下，本项目使用 倾向得分匹配 (PSM) 构造可比对照组，并结合双重差分 (DID) 方法，评估试用会员对用户转化的平均处理效应 (ATT)，为后续 AB 实验设计及 uplift 模型提供依据。

数据准备-- PSM/DID 数据构造

[dataset1_video_app_psm.csv](#)

[dataset2_video_app_did.csv](#)

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from statsmodels.formula.api import ols

# ===== 0. 全局设置 =====
try:
    plt.style.use("seaborn")
except Exception:
    plt.style.use("ggplot")

sns.set_palette("husl")

# ===== 1. 数据准备 =====
print("正在加载数据...")
df1 = pd.read_csv("dataset1_video_app_psm.csv") # 用户特征 + group(0/1)
df2 = pd.read_csv("dataset2_video_app_did.csv", dtype={"date": str}) # 面板数据

# 日期处理：确保 4 位，例如 '0901'
df2["date"] = df2["date"].astype(str).str.zfill(4)

# AA/AB 分期：你原来口径保持不变 (0908 起为 post)
df2["post_period"] = (df2["date"] >= "0908").astype(int)
```

```

print("数据加载成功！")
print(f"数据集1样本量: {len(df1)}")
print(f"数据集2样本量: {len(df2)}")

# ===== 2. PSM 匹配 (with replacement) =====
def psm_matching_with_replacement(data: pd.DataFrame, covariates, k=1, seed=2025):
    """
    倾向得分匹配 (with replacement)
    - 同一个 control 允许被多个 treatment 匹配
    - 通过 NearestNeighbors 在 ps_score 上做最近邻匹配
    - 输出 matched_data 并给出 match_weight 便于后续加权估计 (可选)
    """
    rng = np.random.RandomState(seed)

    df = data.copy()

    # 标准化协变量
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(df[covariates])

    # 估计倾向得分
    ps_model = LogisticRegression(max_iter=2000, random_state=seed)
    ps_model.fit(X_scaled, df["group"])
    df["ps_score"] = ps_model.predict_proba(X_scaled)[:, 1]

    treat = df[df["group"] == 1].copy()
    control = df[df["group"] == 0].copy()

    # 最近邻匹配 (with replacement)
    nbrs = NearestNeighbors(n_neighbors=k, algorithm="ball_tree")
    nbrs.fit(control[["ps_score"]])

    distances, indices = nbrs.kneighbors(treat[["ps_score"]])

    matched_control = control.iloc[indices.flatten()].copy()
    matched_control["matched_id"] = np.repeat(treat.index.values, k)
    matched_control["match_weight"] = 1.0 / k

    treat["matched_id"] = treat.index
    treat["match_weight"] = 1.0

    matched_data = pd.concat([treat, matched_control], axis=0, ignore_index=False)

    return matched_data, ps_model, scaler

# 协变量
covariates = ["gender", "age_group", "play_time"]

print("\n正在执行PSM匹配 (with replacement) ...")
matched_data, ps_model, scaler = psm_matching_with_replacement(df1, covariates,
k=1)

print(
    f"匹配完成: treat={int((matched_data['group']==1).sum())}, "

```

```

    f"matched_control={int((matched_data['group']==0).sum())}"
)

# ===== 3. 匹配平衡性检验 (SMD) =====
def compute_smd(data: pd.DataFrame, covariate: str):
    """
    标准化均值差 (SMD):
    
$$SMD = \frac{\text{mean}_t - \text{mean}_c}{\sqrt{(\text{var}_t + \text{var}_c)/2}}$$

    """
    t = data[data["group"] == 1][covariate].dropna()
    c = data[data["group"] == 0][covariate].dropna()

    mean_t = t.mean()
    mean_c = c.mean()

    var_t = t.var(ddof=1)
    var_c = c.var(ddof=1)
    pooled = np.sqrt((var_t + var_c) / 2.0)

    smd = (mean_t - mean_c) / pooled if pooled > 0 else np.nan
    return mean_t, mean_c, smd

def balance_check_smd(before_df: pd.DataFrame, after_df: pd.DataFrame,
covariates):
    rows_before = []
    rows_after = []

    for cov in covariates:
        mt, mc, smd = compute_smd(before_df, cov)
        rows_before.append([cov, mt, mc, smd])

        mt2, mc2, smd2 = compute_smd(after_df, cov)
        rows_after.append([cov, mt2, mc2, smd2])

    before_tbl = pd.DataFrame(rows_before, columns=["covariate", "mean_treat",
"mean_control", "smd"])
    after_tbl = pd.DataFrame(rows_after, columns=["covariate", "mean_treat",
"mean_control", "smd"])

    print("\n===== 匹配平衡性检验 (SMD) =====")
    print("\n匹配前:")
    print(before_tbl.to_string(index=False))
    print("\n匹配后:")
    print(after_tbl.to_string(index=False))

    # SMD Before vs After 图
    plot_df = pd.DataFrame({
        "covariate": covariates,
        "Before": before_tbl["smd"].values,
        "After": after_tbl["smd"].values
    }).melt(id_vars="covariate", var_name="Stage", value_name="SMD")

    plt.figure(figsize=(10, 5))
    sns.pointplot(data=plot_df, x="SMD", y="covariate", hue="Stage", join=True,

```

```
markers="o")
plt.axvline(0.1, linestyle="--", linewidth=1)
plt.axvline(-0.1, linestyle="--", linewidth=1)
plt.title("Covariate Balance Check: SMD (Before vs After)")
plt.xlabel("Standardized Mean Difference (SMD)")
plt.ylabel("Covariate")
plt.tight_layout()
plt.show()

return before_tbl, after_tbl

before_smd, after_smd = balance_check_smd(df1, matched_data, covariates)
```

匹配检验

----- / -----

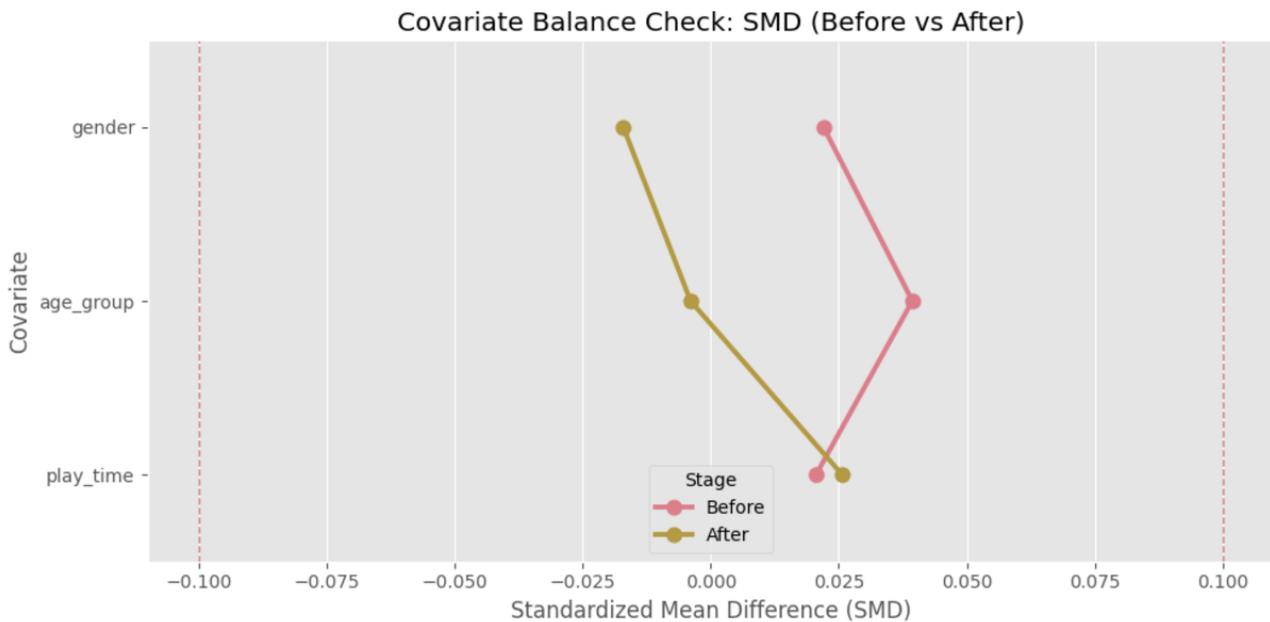
===== 匹配平衡性检验（SMD） =====

匹配前:

covariate	mean_treat	mean_control	smd
gender	0.4056	0.3948	0.022041
age_group	1.6568	1.6284	0.039381
play_time	2309.2252	2298.8680	0.020569

匹配后:

covariate	mean_treat	mean_control	smd
gender	0.4056	0.4140	-0.017077
age_group	1.6568	1.6596	-0.003954
play_time	2309.2252	2296.3472	0.025652



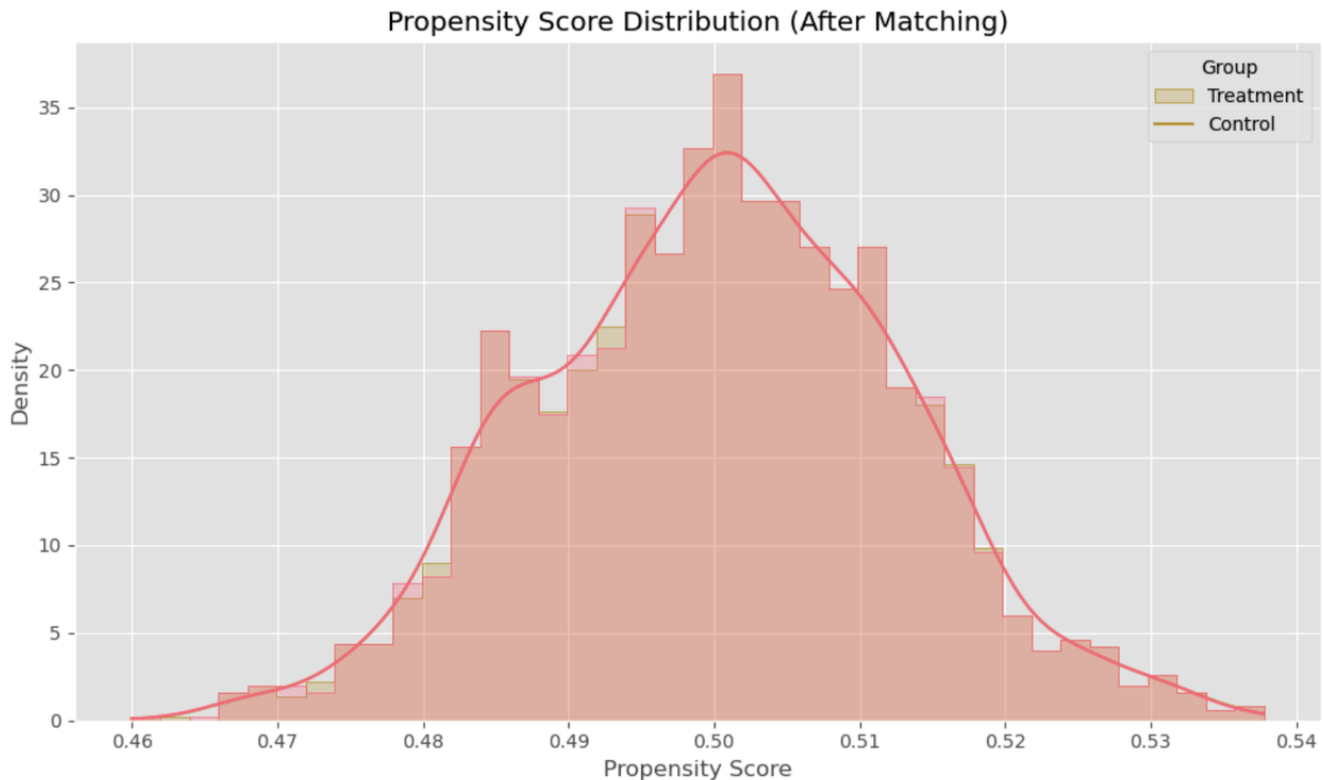
Q: 为什么匹配前和匹配后都比较接近 0? 那你的匹配还有意义吗

A: 匹配前和匹配后都接近 0, 并不异常, 也不说明 PSM 没用。说明在选取的这组协变量上, treatment 与 control 本来就不强烈不平衡。这并不意味着不存在选择偏差, 而是说明偏差主要可能来自未观测变量。

PSM 在这里更多起到的是验证样本可比性的作用, 为后续 DID 的时间差分提供一个更干净的样本基础。

在这里 PSM 的价值, 并不是把严重的不平衡样本拉平, 而是验证在这些可观测的协变量上, treatment/control 是可比的。

```
# ===== 4. 倾向得分分布图 (After Matching) =====
plt.figure(figsize=(10, 6))
sns.histplot(
    data=matched_data,
    x="ps_score",
    hue="group",
    element="step",
    stat="density",
    common_norm=False,
    kde=True
)
plt.title("Propensity Score Distribution (After Matching)")
plt.xlabel("Propensity Score")
plt.ylabel("Density")
plt.legend(title="Group", labels=["Treatment", "Control"])
plt.tight_layout()
plt.show()
```



无论是 SMD 检查 还是 PSD 分布图都可以充分说明，我们构造的实验组和对照组是均匀的。可以用于后续的 DID

```
# ===== 5. 将匹配后的样本映射进 DID 面板 =====
print("\n正在构造 DID 面板 ( 仅保留匹配后的用户 ) ...")

matched_users = matched_data["user_id"].dropna().unique()
df2_matched = df2[df2["user_id"].isin(matched_users)].copy()

# 把 group 补进 df2_matched ( 防止出现 group_x / group_y )
group_map = df1[["user_id", "group"]].drop_duplicates("user_id")

df2_matched = df2_matched.merge(
    group_map,
    on="user_id",
    how="left",
    suffixes=("", "_from_df1")
)

# 兼容：如果 df2 原本就有 group, merge 后会出现 group_from_df1
if "group_from_df1" in df2_matched.columns:
    # 优先用 df1 的 group ( 更可信 )
    df2_matched["group"] = df2_matched["group_from_df1"]
    df2_matched.drop(columns=["group_from_df1"], inplace=True)

# 兜底：如果 merge 失败或 user_id 对不上，直接报更明确的错
if "group" not in df2_matched.columns:
    raise KeyError(
        "df2_matched 中仍然没有 group 列。请检查：df2 是否有 user_id 列、df1 的 user_id 是否能和 df2 对齐。"
```

```

    )

# 基础清洗
df2_matched["group"] = pd.to_numeric(df2_matched["group"],
errors="coerce").fillna(0).astype(int)
df2_matched["post_period"] = df2_matched["post_period"].astype(int)

print(f"DID面板样本量(匹配后用户过滤): {len(df2_matched)}")
print(f"匹配后用户数: {df2_matched['user_id'].nunique()}")
print(df2_matched["group"].value_counts(dropna=False))

# ===== 6. 平行趋势检验 (图 + pre-period 回归) =====
print("\n===== 平行趋势检查 =====")

# 6.1 画每日均值趋势
daily = (
    df2_matched.groupby(["date", "group"])["converted_today"]
    .mean()
    .reset_index()
)

plt.figure(figsize=(10, 5))
sns.lineplot(data=daily, x="date", y="converted_today", hue="group", marker="o")
plt.axvline("0908", linestyle="--", linewidth=1, color="k")
plt.title("Parallel Trend Check (Daily Mean of converted_today)")
plt.xlabel("Date")
plt.ylabel("Mean(converted_today)")
plt.tight_layout()
plt.show()

# 6.2 仅用 pre-period 做回归: converted_today ~ group + time_index +
group:time_index
pre = df2_matched[df2_matched["post_period"] == 0].copy()

# 构造 time_index (按日期排序映射为 0,1,2,...)
sorted_dates = sorted(pre["date"].unique())
date_to_idx = {d: i for i, d in enumerate(sorted_dates)}
pre["time_index"] = pre["date"].map(date_to_idx).astype(int)

pre_model = ols("converted_today ~ group + time_index + group:time_index",
data=pre).fit()

print("\n===== 平行趋势回归检验 (仅 pre-period) =====")
print(pre_model.summary())
print("\n重点看: group:time_index 的系数与 p-value (是否存在差异趋势)")

# ===== 7. DID 模型估计 =====
def compute_att(agg_data: pd.DataFrame):
    pre_t = agg_data[(agg_data.group == 1) & (agg_data.post_period == 0)]
    ["converted_today"].mean()
    post_t = agg_data[(agg_data.group == 1) & (agg_data.post_period == 1)]
    ["converted_today"].mean()
    pre_c = agg_data[(agg_data.group == 0) & (agg_data.post_period == 0)]
    ["converted_today"].mean()

```

```

    post_c = agg_data[(agg_data.group == 0) & (agg_data.post_period == 1)]
    ["converted_today"].mean()
    return (post_t - pre_t) - (post_c - pre_c)

def did_estimation_user_agg(data: pd.DataFrame):
    """
    DID : 按 user_id × post_period 聚合

    """
    agg = data.groupby(["user_id", "post_period"]).agg(
        converted_today=("converted_today", "sum"),
        group=("group", "first")
    ).reset_index()

    agg["treat_post"] = agg["group"] * agg["post_period"]

    model = ols("converted_today ~ group + post_period + treat_post",
data=agg).fit()

    print("\n===== DID OLS 结果 ( user × pre/post 聚合 ) =====")
    print(model.summary())

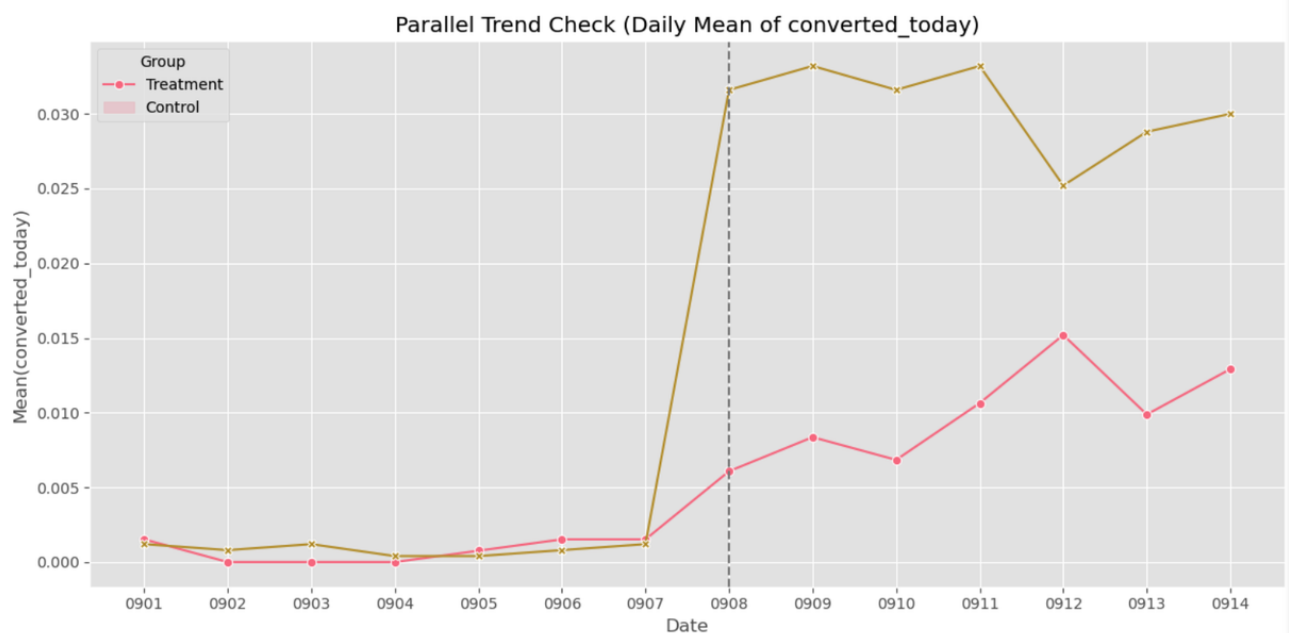
    att = compute_att(agg)
    print(f"\n平均处理效应(ATT): {att:.4f}")

    return model, agg, att

print("\nDID模型估计结果:")
did_model, did_agg, att = did_estimation_user_agg(df2_matched)

```

平行趋势




```
===== 平行趋势回归检验 (仅 pre-period) =====
                                OLS Regression Results
=====
Dep. Variable:      converted_today      R-squared:      0.000
Model:              OLS                  Adj. R-squared:  -0.000
Method:             Least Squares        F-statistic:    0.3211
Date:               Sat, 13 Dec 2025      Prob (F-statistic): 0.810
Time:               22:14:00             Log-Likelihood: 56961.
No. Observations:   26712                AIC:           -1.139e+05
Df Residuals:       26708                BIC:           -1.139e+05
Df Model:           3
Covariance Type:    nonrobust
=====

               coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      0.0004      0.001      0.655      0.513      -0.001      0.001
group          0.0006      0.001      0.886      0.375      -0.001      0.002
time_index     0.0001      0.000      0.908      0.364      -0.000      0.000
group:time_index -0.0002      0.000     -0.890      0.374      -0.001      0.000
=====

Omnibus:          73092.009      Durbin-Watson:      2.002
Prob(Omnibus):    0.000      Jarque-Bera (JB):    1632499552.490
Skew:             34.800      Prob(JB):            0.00
Kurtosis:         1212.095      Cond. No.            21.7
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

重点看: group:time_index 的系数与 p-value (是否存在差异趋势)
```

```
===== DID OLS 结果 (user × pre/post 聚合) =====
                                OLS Regression Results
=====
Dep. Variable:      converted_today      R-squared:      0.095
Model:              OLS                  Adj. R-squared:  0.095
Method:             Least Squares        F-statistic:    268.4
Date:               Sat, 13 Dec 2025      Prob (F-statistic): 1.22e-165
Time:               22:14:00             Log-Likelihood: -1258.2
No. Observations:   7632                AIC:           2524.
Df Residuals:       7628                BIC:           2552.
Df Model:           3
Covariance Type:    nonrobust
=====

               coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      0.0053      0.008      0.676      0.499      -0.010      0.021
group          0.0007      0.010      0.070      0.944      -0.018      0.020
post_period    0.0646      0.011      5.805      0.000      0.043      0.086
treat_post     0.1430      0.014     10.403      0.000      0.116      0.170
=====

Omnibus:          5364.665      Durbin-Watson:      1.992
Prob(Omnibus):    0.000      Jarque-Bera (JB):    78016.913
Skew:             3.290      Prob(JB):            0.00
Kurtosis:         17.214      Cond. No.            8.29
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

平均处理效应(ATT): 0.1430
```

项目流程

step1: 实验准备-

- 实验目的：通过PSM/DID的方式采集策略数据，初步验证策略的效果，为付费人群挖掘提供数据。
- 实验对象：从未完成付费转化的非付费用户。
- 实验设置：
 - 实验组：提供7天限时会员权益。
 - 对照组：无策略
- 观测指标：
 - 核心指标：会员转化 (0-1) is_buy
 - 重要过程指标：用户LT7, 用户播放时长，DAU
- 实验周期：
 - aa实验3天
 - ab实验14天
- 实验结果【数据集1.xlsx】

数据集1.xlsx

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.model_selection import train_test_split
from causalm1.inference.tree import UpliftRandomForestClassifier
from causalm1.metrics import auuc_score
import matplotlib.pyplot as plt

# 1. 数据加载与预处理
ab_display = pd.read_excel('数据集1.xlsx', sheet_name='Sheet1')

# 2. 分类变量编码
le = LabelEncoder()
ab_display['性别'] = le.fit_transform(ab_display['性别'])
ab_display['城市线'] = le.fit_transform(ab_display['城市线'])
ab_display['消费水平'] = le.fit_transform(ab_display['消费水平'])
ab_display['年龄'] = le.fit_transform(ab_display['年龄'])

# OneHot编码垂类
try:
    ohe = OneHotEncoder(sparse_output=False)
except TypeError:
    ohe = OneHotEncoder(sparse=False)

category_ohe = ohe.fit_transform(ab_display[['垂类']])
```

```
category_columns = [f'category_{i}' for i in range(category_ohe.shape[1])]
category_df = pd.DataFrame(category_ohe, columns=category_columns)
ab_display = pd.concat([ab_display, category_df], axis=1).drop('垂类', axis=1)

# 3. 选择特征和目标变量
features = ['性别', '年龄', '消费水平', '城市线'] + category_columns
y = ab_display['is_buy']
treatment = ab_display['组别']
X = ab_display[features]

# 4. 划分训练集和测试集
X_train, X_test, y_train, y_test, treat_train, treat_test = train_test_split(
    X, y, treatment,
    test_size=0.2,
    random_state=42
)

# 5. 模型训练
model = UpliftRandomForestClassifier(
    n_estimators=100,
    random_state=42,
    max_depth=8,
    min_samples_leaf=50,
    control_name='control'
)
model.fit(
    X=X_train.values,
    treatment=treat_train.values,
    y=y_train.values
)

# 6. 预测
y_pred = model.predict(X_test.values)
df_res = pd.DataFrame(y_pred, columns=['pred_lift'])

# 7. 准备评估数据
df_true = pd.DataFrame({
    '组别': treat_test,
    'is_buy': y_test
}).reset_index(drop=True)
df = pd.concat([df_res, df_true], axis=1)

# 8. 将treatment列转换为数值
df['treatment_numeric'] = (df['组别'] == 'treatment').astype(int)

# 9. 计算AUUC
try:
    score = auuc_score(
        df,
        treatment_col='treatment_numeric',
        outcome_col='is_buy',
        uplift_col='pred_lift'
    )
    if isinstance(score, pd.Series):
```

```

        score_value = score.iloc[0]
    else:
        score_value = float(score)
    print(f"AUUC Score: {score_value:.6f}")
except Exception as e:
    print(f"计算AUUC时出错: {e}")
    score_value = 0

# 10. 计算正确的Uplift曲线
def calculate_correct_uplift_curve(df, n_bins=100):
    """计算正确的Uplift曲线"""
    df_sorted = df.sort_values('pred_lift', ascending=False)
    n_samples = len(df_sorted)

    uplift_values = []
    percentages = []

    # 分箱计算，避免数值问题
    for i in range(1, n_bins + 1):
        percent = i * 100 / n_bins
        n_cut = int(n_samples * percent / 100)
        df_subset = df_sorted.head(n_cut)

        # 计算该分箱的Uplift
        treated_subset = df_subset[df_subset['treatment_numeric'] == 1]
        control_subset = df_subset[df_subset['treatment_numeric'] == 0]

        if len(treated_subset) > 0 and len(control_subset) > 0:
            treated_rate = treated_subset['is_buy'].mean()
            control_rate = control_subset['is_buy'].mean()
            uplift = (treated_rate - control_rate) * n_cut # 累积效应
        else:
            uplift = 0

        uplift_values.append(uplift)
        percentages.append(percent)

    return percentages, uplift_values

# 计算正确的曲线
percentages, uplift_values = calculate_correct_uplift_curve(df, n_bins=50)

# 计算随机模型的Uplift曲线 ( 均匀分布 )
random_uplift = np.linspace(0, uplift_values[-1], len(uplift_values))

# 计算整体Uplift ( 用于验证交点 )
overall_treated_rate = df[df['treatment_numeric']==1]['is_buy'].mean()
overall_control_rate = df[df['treatment_numeric']==0]['is_buy'].mean()
overall_uplift = overall_treated_rate - overall_control_rate

print(f"整体Uplift: {overall_uplift:.6f}")

# 绘制仅Normalized Uplift Curve并添加random model
plt.figure(figsize=(10, 6))

```

```
# 标准化处理
if len(uptlift_values) > 0:
    max_uptlift = max(abs(u) for u in uplift_values) or 1
    normalized_uptlift = [u/max_uptlift for u in uplift_values]
    normalized_random = [r/max_uptlift for r in random_uptlift]
else:
    normalized_uptlift = uplift_values
    normalized_random = random_uptlift

# 绘制标准化Uplift曲线
plt.plot(percentages, normalized_uptlift, 'g-', label='Uplift Model', linewidth=2)
plt.plot(percentages, normalized_random, 'r--', label='Random Model', linewidth=2)
plt.axhline(y=0, color='gray', linestyle='-', alpha=0.3)
plt.xlabel('Percentage of Users Targeted (%)')
plt.ylabel('Normalized Uplift')
plt.title('Normalized Uplift Curve Comparison')
plt.legend()
plt.grid(True, alpha=0.3)

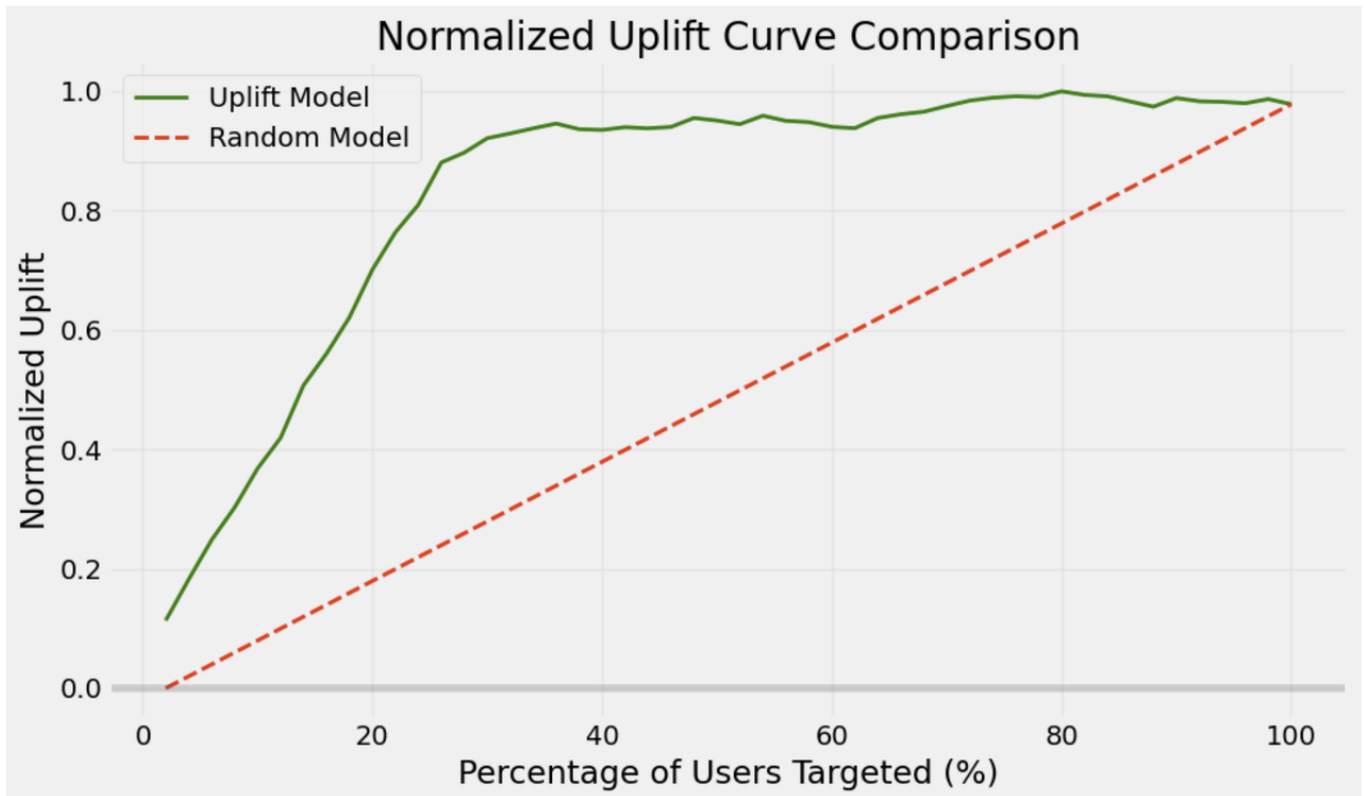
plt.tight_layout()
plt.show()
```

step2: uplift方法圈定潜在正向人群

以用户基本特征和内容偏好特征为输入变量，是否会员转化为响应变量，采用tree-model进行uplift分析，模型auuc score>0.7，说明训练的模型能够有效区分正向用户。

AUUC Score: 0.849734

整体Uplift: 0.031493



【补充了解】 AUUC score 的计算逻辑

```
# 源码逻辑
n_samples = len(df)
n_treatment = sum(df[treatment_col] == 1)
n_control = sum(df[treatment_col] == 0)

# 计算整体转化率
overall_conv_treatment = df[df[treatment_col]==1][outcome_col].mean()
overall_conv_control = df[df[treatment_col]==0][outcome_col].mean()
overall_uplift = overall_conv_treatment - overall_conv_control

# 默认分位数数量 (通常为100)
n_quantiles = min(100, n_samples)

uplift_values = []
random_values = []
for q in np.linspace(0, 1, n_quantiles):
    # 截取前q%样本
    cutoff = int(q * n_samples)
    subset = df.iloc[:cutoff]

    # 计算当前分位内的uplift
    if len(subset) > 0:
        conv_t = subset[subset[treatment_col]==1][outcome_col].mean()
        conv_c = subset[subset[treatment_col]==0][outcome_col].mean()
        uplift = (conv_t - conv_c) * cutoff # 累积效应
    else:
        uplift = 0
```

```
        uplift_values.append(uplift)
        random_values.append(q * overall_uplift * n_samples) # 随机模型的线性增长

# 梯形法计算曲线下面积
auuc = np.trapz(uplift_values, dx=1/n_quantiles) - np.trapz(random_values,
dx=1/n_quantiles)

# 标准化 (可选)
max_possible_area = np.trapz([overall_uplift * n_samples * q for q in
np.linspace(0,1,n_quantiles)], dx=1/n_quantiles)
normalized_auuc = auuc / max_possible_area # 您的0.849734属于此类
```

step3: ab实验检验人群效果

- 实验目的：通过AB检验uplift-model圈定的正向人群是否有效
- 实验设置
 - 实验组：提供7天限时会员权益。
 - 对照组：无策略
- 观测指标：
 - 核心指标：会员转化【0-1】
 - 重要过程指标：人均APP时长，人均播放时长，LT7
- 样本设计：
 - 总体：uplift score排在前20%的用户
- 实验周期：
 - aa实验7天
 - ab实验7天
- 实验结果【数据集2_人群策略_abtest.xlsx】

数据集2_长视频AB_Test.xlsx

	变量	实验组均值	对照组均值	均值差值	t统计量	p值	是否统计显著
0	aa人均播放时长	2034.4574	2029.4390	5.0185	0.8636	0.3878	不显著
1	aa人均app时长	3090.6036	3088.6935	1.9101	0.3015	0.7630	不显著
2	aaCTR	0.4507	0.4512	-0.0005	-0.5016	0.6159	不显著
3	aaLT7	4.0048	4.0003	0.0045	0.5373	0.5911	不显著
4	aaLTV7	6.3674	6.4148	-0.0474	-1.6406	0.1009	不显著
5	aa是否转化	0.0298	0.0288	0.0010	1.1919	0.2333	不显著

aa结果检验，实验组和对照组各个观测指标均没有显著差异。

```
ab_people= pd.read_csv('数据集2_长视频AB_Test.xlsx')

#aa显著性检验
from scipy import stats

# 定义需要进行 t 检验的观测变量
variables = ['aa人均播放时长', 'aa人均app时长', 'aaCTR' , 'aaLT7', 'aaLTV7','aa是否转化']

# 创建一个空的列表，用于存储结果
results = []

# 对每个变量进行 t 检验
for var in variables:
    # 分组：实验组（组别 == 1）和对照组（组别 == 0）
    treatment_group = ab_people[ab_people['组别'] == 'treatment'][var]
    control_group = ab_people[ab_people['组别'] == 'control'][var]
    # 进行双尾 t 检验
    t_stat, p_value = stats.ttest_ind(treatment_group, control_group,
equal_var=False)
    # 计算均值差值
    mean_diff = treatment_group.mean() - control_group.mean()
    # 判断统计显著性
    significance = '显著' if p_value < 0.05 else '不显著'
    # 将结果存储在列表中
    results.append({
        '变量': var,
        '实验组均值': treatment_group.mean(),
        '对照组均值': control_group.mean(),
        '均值差值': mean_diff,
        't统计量': t_stat,
        'p值': p_value,
        '是否统计显著': significance
    })

# 将结果转换为 DataFrame
results_df = pd.DataFrame(results)

# 格式化输出（例如设置浮动数字的显示格式）
results_df = results_df.style.format({
    '实验组均值': '{:.4f}',
    '对照组均值': '{:.4f}',
    '均值差值': '{:.4f}',
    't统计量': '{:.4f}',
    'p值': '{:.4f}'
})

# 输出结果
results_df
```


	变量	实验组均值	对照组均值	均值差值	t统计量	p值	是否统计显著
0	ab人均播放时长	2340.6357	1854.8844	485.7513	78.0896	0.0000	显著
1	ab人均app时长	3025.6234	2814.5137	211.1098	31.5199	0.0000	显著
2	abCTR	0.4904	0.4509	0.0395	37.6212	0.0000	显著
3	abLT7	4.1119	4.0360	0.0759	9.0079	0.0000	显著
4	abLTV7	6.4023	6.3698	0.0325	1.1242	0.2609	不显著
5	ab是否转化	0.0500	0.0308	0.0192	20.2134	0.0000	显著

ab结果检验，实验组和对照组除LTV7均存在显著差异，其中核心是否转化大幅提升，其他指标均为正向提升。实验结果表明人群策略可以显著提升转化率，与业务预期相符。

```
from scipy import stats

# 定义需要进行 t 检验的观测变量
variables = ['ab人均播放时长', 'ab人均app时长', 'abCTR', 'abLT7', 'abLTV7', 'ab是否转化']

# 创建一个空的列表，用于存储结果
results = []

# 对每个变量进行 t 检验
for var in variables:
    # 分组：实验组 (组别 == 'treatment') 和对照组 (组别 == 'control')
    treatment_group = ab_people[ab_people['组别'] == 'treatment'][var]
    control_group = ab_people[ab_people['组别'] == 'control'][var]
    # 进行双尾 t 检验
    t_stat, p_value = stats.ttest_ind(treatment_group, control_group,
    equal_var=False)
    # 计算均值差值
    mean_diff = treatment_group.mean() - control_group.mean()
    # 判断统计显著性
    significance = '显著' if p_value < 0.05 else '不显著'
    # 将结果存储在列表中
    results.append({
        '变量': var,
        '实验组均值': treatment_group.mean(),
        '对照组均值': control_group.mean(),
        '均值差值': mean_diff,
        't统计量': t_stat,
        'p值': p_value,
        '是否统计显著': significance
    })

# 将结果转换为 DataFrame
results_df = pd.DataFrame(results)

# 格式化输出 (例如设置浮动数字的显示格式)
results_df = results_df.style.format({
```

```
'实验组均值': '{:.4f}',  
'对照组均值': '{:.4f}',  
'均值差值': '{:.4f}',  
't统计量': '{:.4f}',  
'p值': '{:.4f}'  
)
```

```
# 输出结果  
results_df
```

项目结论

综上，本项目并未将 PSM、DID、uplift 或 AB 实验视为彼此独立的分析工具，而是将其串联为一条完整的因果验证与策略迭代链路。

首先，在试用会员并非随机触达的现实业务约束下，通过倾向得分匹配（PSM）构造可比的反事实对照组，并结合双重差分（DID）方法，识别试用会员对转化行为的平均处理效应（ATT），验证试用策略在整体层面具备正向因果影响。这一步解决的是“试用是否真的有效”的问题，为后续策略优化提供因果基础。

在确认整体 ATT 为正之后，进一步引入 uplift 建模，刻画试用效果在不同用户之间的异质性，从“是否有效”推进到“对谁更有效”。通过对高 uplift 人群的识别，试用策略不再依赖经验规则，而具备了可量化、可复用的定向投放能力。

最后，通过独立的 AB 实验对 uplift 策略进行线上验证，将模型输出回归到真实业务环境中进行效果评估，从而形成“历史数据因果评估 → 个体异质性建模 → 在线实验验证”的闭环。实验结果表明，相比于非定向或随机投放，基于 uplift 识别的人群在会员使用与转化表现上具备显著优势，验证了 uplift 方法在会员增长场景中的可行性与业务价值。

整体来看，该项目展示了一种在非随机策略环境下，逐步逼近因果真实、并最终服务于可落地增长决策的方法路径，为会员试用策略的精细化运营提供了可扩展的分析范式。